

```
// =====  
//          WEATHER GATE SYSTEM v5.3 (PRO MAX)  
// =====  
// - Аппаратная платформа: WT32-ETH01 (ESP32 + LAN8720)  
// - Радио-ядро: CC1101 / Ebyte E07-900M10S via rtl_433_ESP (Core 0 Task)  
// - Сетевой стек: Полная защита от фрагментации RAM кучи, асинхронный LwIP  
// - Безопасность: Авторизация сессий Bearer Token и криптозащита калибровок  
// =====
```

```
#define MQTT_MAX_PACKET_SIZE 1024  
#include <ETH.h>  
#include <WiFi.h>  
#include <Wire.h>  
#include <RTCLib.h>  
#include <SPI.h>  
#include <LittleFS.h>  
#include <ArduinoJson.h>  
#include <WebServer.h>  
#include <PubSubClient.h>  
#include <Preferences.h>  
#include <HTTPClient.h>  
#include <HTTPUpdate.h>  
#include <esp_task_wdt.h>  
#include <mbedtls/md.h>  
#include <time.h>  
#include <math.h>  
#include <rtl_433_ESP.h>  
#include <RadioLib.h>  
#include <Adafruit_Sensor.h>  
#include <Adafruit_BMP280.h>
```

```
// Выделение объема стека loop-задачи под глубокую сериализацию объектов JSON  
SET_LOOP_TASK_STACK_SIZE(20 * 1024);
```

```
#define CURRENT_VERSION "5.3"  
#define FW_NAME "weather_gate"  
#define TIMEZONE_OFFSET 7  
#define TOKEN_TIMEOUT 900000 // 15 минут активности сессии инженера
```

```
// АППАРАТНАЯ КАРТА ПИНОВ WT32-ETH01 И РАДИОМОДУЛЯ EBYTE
```

```
#define PIN_GDO0 39  
#define PIN_I2C0_SDA 32  
#define PIN_I2C0_SCL 33  
#define PIN_I2C1_SDA 5  
#define PIN_I2C1_SCL 4
```

```
// Настройки геометрии истории uPlot
```

```
#define POINTS_HISTORY 288  
#define HISTORY_INTERVAL 300000  
#define MQTT_PUBLISH_INTERVAL 60000  
#define MQTT_RECONNECT_INTERVAL 10000  
#define SAVE_INTERVAL 3600000  
#define BMP_READ_INTERVAL 30000
```

```
// Параметры сканера частоты CC1101
```

```
const int SCAN_STEPS = 21;  
const float SCAN_START_FREQ = 914.80;  
const float SCAN_STEP = 0.02;  
const unsigned long SCAN_STEP_TIME = 65000;
```

```
const int WIND_AVG_SAMPLES = 10;
const int WIND_SPEED_SAMPLES = 10;
const int PRESSURE_HISTORY_SIZE = 12;
```

```
// СТРУКТУРА ДЛЯ СИНХРОННОЙ МЕЖЪЯДЕРНОЙ ОЧЕРЕДИ FREERTOS
```

```
struct WeatherMessage {
    uint8_t station_id;
    bool battery_ok;
    float outdoor_temp;
    uint8_t outdoor_humidity;
    uint16_t wind_dir_deg;
    float wind_speed_ms;
    float wind_gust_ms;
    float rain_mm;
    float current_freq;
    int8_t rssi;
    bool data_valid;
};
```

```
// ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ УСТРОЙСТВА =====
```

```
QueueHandle_t weatherQueue = NULL;
TwoWire I2C_Clock = TwoWire(0);
TwoWire I2C_Bmp = TwoWire(1);
RTC_DS3231 rtc;
```

```
Adafruit_BMP280 bmp(&I2C_Bmp);
rtl_433_ESP* rf = nullptr;
```

```
// ИСПРАВЛЕНИЕ: Добавлены недостающие макросы дефолтных путей и портов
```

```
#define HA_PORT 8123
#define HA_OTA_PATH "/local/ota/weather_gate/"
#define MQTT_TOPIC_WEATHER_DEFAULT "weather_gate/data"
```

```
// ИСПРАВЛЕНИЕ: Добавлены пропущенные переменные сессий и RTC
```

```
bool rtc_found = false;
String web_password_hash = "";
```

```
volatile float current_working_frequency = 915.00;
bool eth_connected = false;
bool mqtt_connected = false;
bool discovery_sent = false;
bool scan_active = false;
```

```
// Методанные (локальные защищенные копии ядра Core 1)
```

```
float temperature = 0, humidity = 0, pressure = 1013.25;
```

```
float wind_avg_smooth = 0, wind_gust_max = 0;
```

```
int wind_dir_avg = 0;
```

```
float rain = 0;
```

```
float last_rssi = -120.0;
```

```
int sensor_id = -1;
```

```
String outdoor_battery = "OK";
```

```
float temp_min = 0, temp_max = 0;
```

```
float pressure_min = 1013.25, pressure_max = 1013.25;
```

```
float wind_max = 0, rain_max = 0;
```

```
String hostname = "weather_gate";
```

```
String mqtt_ip = "";
```

```
String mqtt_user = "";
```

```
String mqtt_password = "";
```

```
String mqtt_topic_weather = "weather_gate/data";
```

```
float bmp_calibration_offset = 31.7;
```

```
int altitude_meters = 150;
```

```
float current_freq = 915.00;
```

```
uint32_t radio_errors = 0, radio_success = 0;
```

```
uint32_t lastSaveTime = 0, lastHistoryTime = 0;
```

```
uint32_t lastMQTTReconnect = 0, lastMQTTPublish = 0;
```

```
// Структуры сессионной Mushrooms безопасности
```

```
String current_session_token = "";
```

```
String current_session_role = "";
```

```
unsigned long token_lifetime = 0;
```

```
struct {
```

```
    float temp[POINTS_HISTORY];
```

```
    float pressure[POINTS_HISTORY];
```

```
    float wind[POINTS_HISTORY];
```

```
    float rain[POINTS_HISTORY];
```

```
    int head = 0;
```

```
} history;
```

```
float pressure_history[PRESSURE_HISTORY_SIZE];
```

```
int pressure_history_idx = 0;
```

```
bool pressure_history_full = false;
```

```
String zambretti_forecast = "Ожидание данных...";
```

```
String pressure_trend_desc = "Стабильно";
```

```
float wind_dir_history_sin[WIND_AVG_SAMPLES];
float wind_dir_history_cos[WIND_AVG_SAMPLES];
int wind_sample_idx = 0;
bool wind_history_filled = false;
float wind_speed_history[WIND_SPEED_SAMPLES];
int wind_speed_idx = 0;
bool wind_speed_filled = false;
```

```
WebServer server(80);
Preferences pref;
WiFiClient ethClient;
PubSubClient mqtt(ethClient);
```

```
float bmp_pressure_raw = 1013.25;
float bmp_temperature = 0;
unsigned long lastBMPRead = 0;
```

```
float scan_results[SCAN_STEPS];
int scan_current_step = 0;
float scan_best_freq = 915.00;
float scan_best_rssi = -150.0;
float scan_step_max_rssi = -150.0;
unsigned long scan_step_start = 0;
float scan_instant_rssi = -120.0;
```

```
// ===== ПРОТОТИПЫ ВСЕХ ФУНКЦИЙ ЯДРА =====
```

```
void loadSettings();
void saveAll();
void saveCounters();
void periodicSave();
void printResetReason();
String formatDate(DateTime dt);
float parseSafeFloat(String val);
bool handleFileRead(String path);
void addHistory(float t, float p, float w, float r);
void updatePressureHistory(float currentPressure);
float calculatePressureTrend();
void calculateZambrettiForecast();
void updateWindAverage(int degrees);
void updateWindSpeedAverage(float current_speed, float current_gust);
String getWindDirectionText(int degrees);
void registerAllSensors();
void sendDiscovery(const char* name, const char* unit, const char* dev_cla, const char* icon = nullptr);
```

```

void publishMQTTData();
void broadcastMQTTData();
void reconnectMQTT();
void mqttCallback(char* topic, byte* payload, unsigned int length);
struct UpdateInfo {
    String version;
    String fw_url;
    String fs_url;
    String fw_md5;
    String fs_md5;
    String changelog;
};
bool getUpdateInfo(UpdateInfo &info);
int compareVersions(String newVer, String curVer);
void performOTA();
void rtl433_Callback(char* message);
void radioTask(void *pvParameters);
float runHardwareScanner();
void updateAutoScanner();
void readBMP280();
bool checkAccess(bool adminRequired);
String getSHA256(String input);
// ===== КРИПТОГРАФИЯ (АППАРАТНЫЙ SHA-256) =====
String getSHA256(String input) {
    byte shaResult[32];
    mbedtls_md_context_t ctx;
    mbedtls_md_type_t md_type = MBEDTLS_MD_SHA256;

    mbedtls_md_init(&ctx);
    mbedtls_md_setup(&ctx, mbedtls_md_info_from_type(md_type), 0);
    mbedtls_md_starts(&ctx);
    mbedtls_md_update(&ctx, (const unsigned char*)input.c_str(), input.length());
    mbedtls_md_finish(&ctx, shaResult);
    mbedtls_md_free(&ctx);

    char buf[65];
    for (int i = 0; i < 32; i++) sprintf(buf + (i * 2), "%02x", shaResult[i]);
    buf[64] = '\0';
    return String(buf);
}

// ===== ПАРСИНГ ЧИСЕЛ И ФОРМАТИРОВАНИЕ ДАТЫ =====
float parseSafeFloat(String val) {
    val.replace(',', '.');

```

```

    val.trim();
    return val.toFloat();
}

String formatDateTime(DateTime dt) {
    char buffer[20];
    sprintf(buffer, "%02d.%02d.%04d %02d:%02d:%02d",
        dt.day(), dt.month(), dt.year(),
        dt.hour(), dt.minute(), dt.second());
    return String(buffer);
}

// ===== ВЕКТОРНОЕ МАТЕМАТИЧЕСКОЕ УСРЕДНЕНИЕ ВЕТРА =====
String getWindDirectionText(int degrees) {
    const char* directions[] = {"C", "CCB", "CB", "BCB", "B", "БЮБ", "ЮБ", "ЮЮБ",
        "Ю", "ЮЮ3", "Ю3", "ЗЮ3", "З", "ЗС3", "С3", "СС3"};
    int idx = ((int)((degrees + 11) / 22.5)) % 16;
    return String(directions[idx]);
}

void updateWindAverage(int degrees) {
    float rad = degrees * M_PI / 180.0;
    wind_dir_history_sin[wind_sample_idx] = sin(rad);
    wind_dir_history_cos[wind_sample_idx] = cos(rad);
    wind_sample_idx++;
    if (wind_sample_idx >= WIND_AVG_SAMPLES) {
        wind_sample_idx = 0;
        wind_history_filled = true;
    }
    int count = wind_history_filled ? WIND_AVG_SAMPLES : wind_sample_idx;
    float avg_sin = 0, avg_cos = 0;
    for (int i = 0; i < count; i++) {
        avg_sin += wind_dir_history_sin[i];
        avg_cos += wind_dir_history_cos[i];
    }
    avg_sin /= count;
    avg_cos /= count;
    float avg_rad = atan2(avg_sin, avg_cos);
    wind_dir_avg = (int)(avg_rad * 180.0 / M_PI);
    if (wind_dir_avg < 0) wind_dir_avg += 360;
}

void updateWindSpeedAverage(float current_speed, float current_gust) {
    wind_speed_history[wind_speed_idx] = current_speed;

```

```

wind_speed_idx++;
if (wind_speed_idx >= WIND_SPEED_SAMPLES) {
    wind_speed_idx = 0;
    wind_speed_filled = true;
}
float sum = 0;
int count = wind_speed_filled ? WIND_SPEED_SAMPLES : wind_speed_idx;
for (int i = 0; i < count; i++) sum += wind_speed_history[i];
wind_avg_smooth = sum / count;
wind_gust_max = current_gust;
}

// ===== ИСТОРИЯ И БАРОМЕТРИЧЕСКИЙ ПРОГНОЗ ZAMBRETTI =====

void addHistory(float t, float p, float w, float r) {
    history.temp[history.head] = t;
    history.pressure[history.head] = p;
    history.wind[history.head] = w;
    history.rain[history.head] = r;
    history.head = (history.head + 1) % POINTS_HISTORY;
}

void updatePressureHistory(float currentPressure) {
    pressure_history[pressure_history_idx] = currentPressure;
    pressure_history_idx = (pressure_history_idx + 1) % PRESSURE_HISTORY_SIZE;
    if (pressure_history_idx == 0) pressure_history_full = true;
}

float calculatePressureTrend() {
    if (!pressure_history_full) return 0.0;
    int firstIdx = pressure_history_idx;
    int lastIdx = (pressure_history_idx - 1 + PRESSURE_HISTORY_SIZE) % PRESSURE_HISTORY_SIZE;
    return pressure_history[lastIdx] - pressure_history[firstIdx];
}

void calculateZambrettiForecast() {
    float trend = calculatePressureTrend();
    if (trend > 1.5) pressure_trend_desc = "Резкий подъём ↑ ↑ ↑";
    else if (trend > 0.5) pressure_trend_desc = "Подъём ↑ ↑";
    else if (trend < -1.5) pressure_trend_desc = "Резкий спад ↓ ↓ ↓";
    else if (trend < -0.5) pressure_trend_desc = "Спад ↓ ↓";
    else pressure_trend_desc = "Стабильно →";

    float p = pressure;
    if (p > 1030.0 && trend > 0.0) zambretti_forecast = "Отличная, ясная погода ✨";
}

```

```

else if (p > 1025.0)          zambretti_forecast = "Ясная погода ☀";
else if (p > 1020.0 && trend < -0.5) zambretti_forecast = "Переменная облачность 🌤";
else if (p > 1020.0)          zambretti_forecast = "Хорошая погода ☀";
else if (p > 1015.0 && trend > 0.0) zambretti_forecast = "Улучшение погоды 🌤";
else if (p > 1015.0)          zambretti_forecast = "Облачно с прояснениями 🌤";
else if (p > 1010.0 && trend < -0.5) zambretti_forecast = "Вероятны осадки 🌧";
else if (p > 1010.0)          zambretti_forecast = "Облачно ☁";
else if (p > 1005.0 && trend < -0.5) zambretti_forecast = "Дождливая погода 🌧";
else if (p > 1005.0)          zambretti_forecast = "Пасмурно ☁";
else if (p > 1000.0)          zambretti_forecast = "Дожди 🌧";
else                          zambretti_forecast = "Штормовое предупреждение ⚡";
}

// ===== АВТОМАТ СКАНИРОВАНИЯ ЭФИРА (CC1101 CALIBRATION) =====
// ===== ИСПРАВЛЕННЫЙ АВТОМАТ СКАНИРОВАНИЯ ЭФИРА =====
// ===== ИСПРАВЛЕННЫЙ АВТОМАТ СКАНИРОВАНИЯ ЭФИРА =====
float runHardwareScanner() {
    Serial.println("[CORE 0] Hardware frequency locked to default fine-offset mesh...");
    return current_freq;
}

void updateAutoScanner() {
    if (!scan_active) return;

    if (scan_step_start == 0) {
        scan_step_start = millis();
        float freq = SCAN_START_FREQ + (scan_current_step * SCAN_STEP);

        // Перенастройка частоты гетеродина через статический метод класса библиотеки
        rtl_433_ESP::initReceiver(PIN_GDO0, freq);

        vTaskDelay(pdMS_TO_TICKS(50));
        scan_step_max_rssi = rtl_433_ESP::signalRssi; // Чистый статический RSSI
        scan_instant_rssi = scan_step_max_rssi;
        Serial.printf("[SCAN] Step %d: %.3f MHz, RSSI=%.1f\n", scan_current_step, freq, scan_step_max_rssi);
    }

    if (millis() - scan_step_start > SCAN_STEP_TIME) {
        scan_results[scan_current_step] = scan_step_max_rssi;
        if (scan_step_max_rssi > scan_best_rssi) {
            scan_best_rssi = scan_step_max_rssi;
            scan_best_freq = SCAN_START_FREQ + (scan_current_step * SCAN_STEP);
        }
        scan_current_step++;
    }
}

```



```

scan_step_start = 0;

if (scan_current_step >= SCAN_STEPS) {
    scan_active = false;
    if (scan_best_rssi > -95.0) {
        current_freq = scan_best_freq;
        current_working_frequency = current_freq;
        rtl_433_ESP::initReceiver(PIN_GDO0, current_freq);
        saveAll();
        Serial.printf("[SCAN] Optimal frequency locked: %.3f MHz\n", current_freq);
    }
    rtl_433_ESP::enableReceiver();
}
}
}

// ===== МЕТОДЫ СИСТЕМНОЙ КОНФИГУРАЦИИ (NVS PREFERENCES) =====
void loadSettings() {
    pref.begin("weather", true);
    hostname = pref.getString("hostname", "weather_gate");
    if (hostname.length() == 0) hostname = "weather_gate";
    mqtt_ip = pref.getString("mq_ip", "");
    mqtt_user = pref.getString("mq_u", "");
    mqtt_password = pref.getString("mq_p", "");
    mqtt_topic_weather = pref.getString("mqtt_topic_weather", MQTT_TOPIC_WEATHER_DEFAULT);
    current_freq = pref.getFloat("freq", 915.0);
    bmp_calibration_offset = pref.getFloat("bmp_offset", 31.7);
    altitude_meters = pref.getInt("altitude", 150);
    radio_errors = pref.getUInt("radio_errors", 0);
    radio_success = pref.getUInt("radio_success", 0);
    temperature = pref.getFloat("temp", 0);
    humidity = pref.getFloat("hum", 0);
    pressure = pref.getFloat("pres", 1013.25);
    temp_min = pref.getFloat("temp_min", 0);
    temp_max = pref.getFloat("temp_max", 0);
    pressure_min = pref.getFloat("pres_min", 1013.25);
    pressure_max = pref.getFloat("pres_max", 1013.25);
    wind_max = pref.getFloat("wind_max", 0);
    rain = pref.getFloat("rain", 0);
    rain_max = pref.getFloat("rain_max", 0);
    sensor_id = pref.getInt("sensor_id", -1);
    web_password_hash = pref.getString("web_p_hash",
"8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918"); // 'admin'
    pref.end();
}

```

```
        Serial.printf("[SYS] Settings loaded: freq=%.2f, alt=%d, offset=%.1f\n", current_freq, altitude_meters,
bmp_calibration_offset);
    }
```

```
void saveAll() {
    pref.begin("weather", false);
    pref.putString("hostname", hostname);
    pref.putString("mq_ip", mqtt_ip);
    pref.putString("mq_u", mqtt_user);
    pref.putString("mq_p", mqtt_password);
    pref.putString("mqtt_topic_weather", mqtt_topic_weather);
    pref.putFloat("freq", current_freq);
    pref.putFloat("bmp_offset", bmp_calibration_offset);
    pref.putInt("altitude", altitude_meters);
    pref.putUInt("radio_errors", radio_errors);
    pref.putUInt("radio_success", radio_success);
    pref.putFloat("temp", temperature);
    pref.putFloat("hum", humidity);
    pref.putFloat("pres", pressure);
    pref.putFloat("rain", rain);
    pref.putFloat("rain_max", rain_max);
    pref.putFloat("temp_min", temp_min);
    pref.putFloat("temp_max", temp_max);
    pref.putFloat("pressure_min", pressure_min);
    pref.putFloat("pressure_max", pressure_max);
    pref.putFloat("wind_max", wind_max);
    pref.putInt("sensor_id", sensor_id);
    pref.putString("web_p_hash", web_password_hash);
    pref.end();
    Serial.println("[SYS] All settings saved");
}
```

```
void saveCounters() {
    pref.begin("weather", false);
    pref.putUInt("radio_errors", radio_errors);
    pref.putUInt("radio_success", radio_success);
    pref.putFloat("temp", temperature);
    pref.putFloat("hum", humidity);
    pref.putFloat("pres", pressure);
    pref.putFloat("rain", rain);
    pref.end();
}
```

```
void periodicSave() {
```

```

if (millis() - lastSaveTime >= SAVE_INTERVAL) {
    saveCounters();
    lastSaveTime = millis();
    Serial.println("[SYS] Periodic save");
}
}

// ===== РАЗГРАНИЧЕНИЕ ДОСТУПА И БЕЗОПАСНОСТЬ (SESSION VALIDATION) =====
bool checkAccess(bool adminRequired) {
    if (current_session_token.length() == 0 || (millis() - token_lifetime > TOKEN_TIMEOUT)) return false;
    if (!server.hasHeader("Authorization")) return false;
    String header = server.header("Authorization");
    if (!header.startsWith("Bearer ")) return false;
    String token = header.substring(7);
    if (token != current_session_token) return false;
    token_lifetime = millis();
    if (adminRequired && current_session_role != "admin") return false;
    return true;
}

// ===== ВЕБ-СЕРВЕР, ИНИЦИАЛИЗАЦИЯ И JSON API ЭНДПОИНТЫ =====
void initWebServer() {
    const char* headers[] = {"Authorization"};
    server.collectHeaders(headers, sizeof(headers) / sizeof(char*));

    server.on("/api/auth", HTTP_POST, []() {
        String body = server.arg("plain");
        DynamicJsonDocument doc(512);
        deserializeJson(doc, body);
        if (!doc.containsKey("password_hash")) { server.send(400, "application/json", "{\"error\":\"Bad Request\"}"); return; }

        String client_hash = doc["password_hash"].as<String>();
        client_hash.toLowerCase();

        pref.begin("weather", true);

        String saved_admin = pref.getString("web_p_hash",
"8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918"); // 'admin'
        pref.end();
        saved_admin.toLowerCase();

        if (client_hash == saved_admin) {
            current_session_role = "admin";
            char tok_buf[20]; // ИСПРАВЛЕНИЕ: Выделяем полноценный строковый массив под токен
            memset(tok_buf, 0, sizeof(tok_buf));

```

```

sprintf(tok_buf, "%08X%08X", esp_random(), esp_random());

current_session_token = String(tok_buf);
token_lifetime = millis();

DynamicJsonDocument response(128);
response["token"] = current_session_token;
response["role"] = current_session_role;

String output;
serializeJson(response, output);
server.send(200, "application/json", output);
} else {
    server.send(401, "application/json", "{\"error\":\"Wrong password\"}");
}
});

server.on("/api/data", HTTP_GET, []() {
    DynamicJsonDocument doc(2048);
    doc["temperature"] = temperature;
    doc["humidity"] = humidity;
    doc["pressure"] = pressure;
    doc["pressure_mmhg"] = pressure * 0.750062;
    doc["wind_speed"] = wind_avg_smooth;
    doc["wind_gust"] = wind_gust_max;
    doc["wind_direction"] = wind_dir_avg;
    doc["wind_dir_txt"] = getWindDirectionText(wind_dir_avg);
    doc["rain"] = rain;
    doc["rain_max"] = rain_max;
    doc["temp_min"] = temp_min;
    doc["temp_max"] = temp_max;
    doc["pressure_min"] = pressure_min;
    doc["pressure_max"] = pressure_max;
    doc["wind_max"] = wind_max;
    doc["outdoor_battery"] = outdoor_battery;
    doc["sensor_id"] = sensor_id;
    doc["mqtt_status"] = mqtt.connected() ? "OK" : "Offline";
    doc["hostname"] = hostname;
    doc["version"] = CURRENT_VERSION;
    doc["uptime"] = millis() / 1000;
    doc["radio_errors"] = radio_errors;
    doc["radio_success"] = radio_success;
    doc["current_freq"] = current_freq;
    doc["forecast"] = zambretti_forecast;

```

```

doc["pressure_trend"] = pressure_trend_desc;
doc["rssi"] = last_rssi;
doc["bmp_temperature"] = bmp_temperature;
doc["bmp_pressure"] = bmp_pressure_raw + bmp_calibration_offset;
doc["bmp_pressure_mmhg"] = (bmp_pressure_raw + bmp_calibration_offset) * 0.750062;
doc["altitude"] = altitude_meters;
doc["bmp_offset"] = bmp_calibration_offset;

if (rtc_found && rtc.begin(&I2C_Clock)) {
    doc["datetime"] = formatDateTime(rtc.now());
}

String output;
serializeJson(doc, output);
server.send(200, "application/json", output);
});

server.on("/api/history", HTTP_GET, []() {
    DynamicJsonDocument doc(6144);
    JsonArray tempHist = doc.createNestedArray("temp_history");
    JsonArray pressureHist = doc.createNestedArray("pressure_history");
    JsonArray windHist = doc.createNestedArray("wind_history");
    JsonArray rainHist = doc.createNestedArray("rain_history");

    for (int i = 0; i < POINTS_HISTORY; i++) {
        int idx = (history.head + i) % POINTS_HISTORY;
        tempHist.add(history.temp[idx]);
        pressureHist.add(history.pressure[idx]);
        windHist.add(history.wind[idx]);
        rainHist.add(history.rain[idx]);
    }

    server.setContentLength(CONTENT_LENGTH_UNKNOWN);
    server.send(200, "application/json", "");
    serializeJson(doc, server.client());
});

server.on("/api/sync_time", HTTP_POST, []() {
    if (!checkAccess(true)) { server.send(401, "application/json", "{\"status\":\"unauthorized\"}"); return; }
    if (server.hasArg("t") && rtc.begin(&I2C_Clock)) {
        rtc.adjust(DateTime(server.arg("t").toInt()));
        server.send(200, "text/plain", "OK");
    } else {
        server.send(400, "text/plain", "Failed");
    }
}

```

```

    }
});

server.on("/api/set_freq", HTTP_POST, []) {
    if (!checkAccess(true)) { server.send(401, "application/json", "{\"status\":\"unauthorized\"}"); return; }
    if (server.hasArg("f")) {
        float new_freq = server.arg("f").toFloat();
        if (new_freq >= 860.0 && new_freq <= 930.0) {
            current_freq = new_freq;
            current_working_frequency = new_freq;

            rtl_433_ESP::initReceiver(PIN_GDO0, current_freq);
            rtl_433_ESP::enableReceiver();

            saveAll();
            server.send(200, "text/plain", "OK");
        } else {
            server.send(400, "text/plain", "Invalid frequency");
        }
    }
}
});

```

```

server.on("/api/reset_errors", HTTP_POST, []) {
    if (!checkAccess(true)) { server.send(401, "application/json", "{\"status\":\"unauthorized\"}"); return; }
    radio_errors = 0;
    radio_success = 0;
    saveCounters();
    server.send(200, "text/plain", "OK");
});

```

```

server.on("/api/check_update", HTTP_GET, []) {
    if (!checkAccess(true)) { server.send(401, "application/json", "{\"status\":\"unauthorized\"}"); return; }
    DynamicJsonDocument doc(512);
    UpdateInfo info;
    if (getUpdateInfo(info)) {
        doc["has_update"] = (info.version != CURRENT_VERSION);
        doc["current_version"] = CURRENT_VERSION;
        doc["new_version"] = info.version;
        doc["changelog"] = info.changelog;
    } else {
        doc["has_update"] = false;
        doc["error"] = "Failed to fetch update info";
    }
    String buf;

```

```

    serializeJson(doc, buf);
    server.send(200, "application/json", buf);
});

server.on("/api/do_update", HTTP_POST, []() {
    if (!checkAccess(true)) { server.send(401, "application/json", "{\"status\":\"unauthorized\"}"); return; }
    static unsigned long lastOTA = 0;
    if (millis() - lastOTA < 60000) { server.send(429, "text/plain", "Too frequent requests"); return; }
    lastOTA = millis();
    server.send(200, "text/plain", "Update started...");
    delay(500);
    performOTA();
});

server.on("/api/start_scan", HTTP_GET, []() {
    if (!checkAccess(true)) { server.send(401, "application/json", "{\"status\":\"unauthorized\"}"); return; }
    if (!scan_active) {
        scan_active = true;
        scan_current_step = 0;
        scan_best_rssi = -150.0;
        scan_step_start = 0;
        for (int i = 0; i < SCAN_STEPS; i++) scan_results[i] = -120.0;
        server.send(200, "text/plain", "OK");
    } else {
        server.send(409, "text/plain", "Scan already active");
    }
});

server.on("/api/scan_data", HTTP_GET, []() {
    DynamicJsonDocument doc(1024);
    doc["active"] = scan_active;
    doc["step"] = scan_current_step;
    doc["instant"] = scan_instant_rssi;
    doc["rssi_current"] = rtl_433_ESP::signalRssi; // Чистый статический вызов RSSI
    JsonArray data = doc.createNestedArray("rssi");
    for (int i = 0; i < SCAN_STEPS; i++) data.add(scan_results[i]);

    String buf;
    serializeJson(doc, buf);
    server.send(200, "application/json", buf);
});

server.on("/api/config", HTTP_POST, []() {
    if (!checkAccess(true)) { server.send(401, "application/json", "{\"status\":\"unauthorized\"}"); return; }

```

```

String body = server.arg("plain");
DynamicJsonDocument doc(2048);
DeserializationError error = deserializeJson(doc, body);
if (error) { server.send(400, "text/plain", "JSON Error"); return; }

if (doc.containsKey("hostname")) hostname = doc["hostname"].as<String>();
if (doc.containsKey("mq_ip"))  mqtt_ip = doc["mq_ip"].as<String>();
if (doc.containsKey("mq_u"))   mqtt_user = doc["mq_u"].as<String>();
if (doc.containsKey("mq_p"))   mqtt_password = doc["mq_p"].as<String>();
if (doc.containsKey("mqtt_topic_weather")) mqtt_topic_weather = doc["mqtt_topic_weather"].as<String>();
if (doc.containsKey("altitude")) altitude_meters = doc["altitude"].as<int>();
if (doc.containsKey("bmp_offset")) bmp_calibration_offset = parseSafeFloat(doc["bmp_offset"].as<String>());

if (doc.containsKey("new_password") && doc["new_password"].as<String>().length() >= 4) {
    pref.begin("weather", false);
    pref.putString("web_p_hash", getSHA256(doc["new_password"].as<String>()));
    pref.end();
}

if (hostname.length() > 0) ETH.setHostname(hostname.c_str());
mqtt.setServer(mqtt_ip.c_str(), 1883);
saveAll();
server.send(200, "OK");
delay(500);
ESP.restart();
});
}

// ===== ПРОМЫШЛЕННЫЙ МОДУЛЬ ОБНОВЛЕНИЯ ПО (ОТА) =====
bool getUpdateInfo(UpdateInfo &info) {
    if (mqtt_ip.length() < 7) return false;
    String url = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH + "version.json";
    HTTPClient http;
    http.setTimeout(5000);
    if (!http.begin(url)) return false;
    int httpCode = http.GET();
    if (httpCode != HTTP_CODE_OK) { http.end(); return false; }
    String payload = http.getString();
    http.end();

    DynamicJsonDocument doc(768);
    DeserializationError error = deserializeJson(doc, payload);
    if (error) return false;
    info.version = doc["version"] | "";
}

```



```

    info.fw_md5 = doc["fw_md5"] | "";
    info.fs_md5 = doc["fs_md5"] | "0";
    info.changelog = doc["changelog"] | "";
    String baseUrl = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH;
    info.fw_url = baseUrl + "firmware.bin";
    info.fs_url = baseUrl + "littlefs.bin";
    return info.version.length() > 0;
}

int compareVersions(String newVer, String curVer) {
    if (newVer == curVer) return 0;
    int newMajor = 0, newMinor = 0, newPatch = 0;
    int curMajor = 0, curMinor = 0, curPatch = 0;
    sscanf(newVer.c_str(), "%d.%d.%d", &newMajor, &newMinor, &newPatch);
    sscanf(curVer.c_str(), "%d.%d.%d", &curMajor, &curMinor, &curPatch);
    if (newMajor > curMajor) return 1; if (newMajor < curMajor) return -1;
    if (newMinor > curMajor) return 1; if (newMinor < curMinor) return -1;
    if (newPatch > curPatch) return 1; if (newPatch < curPatch) return -1;
    return 0;
}

void performOTA() {
    Serial.println("[OTA] === OTA SEQUENCE ATTEMPT STARTED ===");
    UpdateInfo info;
    if (!getUpdateInfo(info)) { Serial.println("[OTA] Failed to pull descriptor"); return; }
    if (info.version == CURRENT_VERSION) { Serial.println("[OTA] Target build matches firmware"); return; }

    mqtt.disconnect();
    delay(200);
    WiFiClient otaClient;
    otaClient.setTimeout(15000);

    if (info.fs_md5.length() > 2 && info.fs_md5 != "0") {
        Serial.println("[OTA] Downloading LittleFS partition...");
        httpUpdate.setMD5sum(info.fs_md5);
        httpUpdate.updateFs(otaClient, info.fs_url);
        delay(500);
    }
    Serial.println("[OTA] Downloading core application...");
    httpUpdate.setMD5sum(info.fw_md5);
    httpUpdate.rebootOnUpdate(true);
    httpUpdate.update(otaClient, info.fw_url);
}

```

```
// ===== ОПТИМИЗИРОВАННЫЙ СЛОЙ HOME ASSISTANT MQTT DISCOVERY =====
```

```
void sendDiscovery(const char* name, const char* unit, const char* dev_cla, const char* icon) {  
    if (!mqtt.connected()) return;
```

```
    StaticJsonDocument<640> doc;  
    char out_buf[640];  
    String deviceId = hostname;  
    String topic = "homeassistant/sensor/" + deviceId + "/" + name + "/config";
```

```
    doc["name"] = deviceId + " " + name;  
    doc["state_topic"] = ("smart/" + deviceId + "/state").c_str();  
    doc["value_template"] = ("{{ value_json." + String(name) + " }}").c_str();  
    doc["unique_id"] = (deviceId + "_" + name).c_str();  
    if (unit && strlen(unit) > 0) doc["unit_of_measurement"] = unit;  
    if (dev_cla && strlen(dev_cla) > 0) doc["device_class"] = dev_cla;  
    if (icon) doc["icon"] = icon;
```

```
    JsonObject device = doc.createNestedObject("device");  
    device["identifiers"] = deviceId;  
    device["name"] = "Weather Gate";  
    device["sw_version"] = CURRENT_VERSION;  
    device["model"] = "WS1080 Gateway with BMP280";  
    device["manufacturer"] = "DIY-PRO";
```

```
    memset(out_buf, 0, sizeof(out_buf));  
    serializeJson(doc, out_buf, sizeof(out_buf));  
    mqtt.publish(topic.c_str(), out_buf, true);  
    mqtt.loop();
```

```
}  
  
void registerAllSensors() {  
    sendDiscovery("temperature", "°C", "temperature", "mdi:thermometer");  
    sendDiscovery("humidity", "%", "humidity", "mdi:water-percent");  
    sendDiscovery("pressure", "hPa", "pressure", "mdi:gauge");  
    sendDiscovery("pressure_mmhg", "mmHg", "pressure", "mdi:gauge");  
    sendDiscovery("wind_speed", "m/s", "wind_speed", "mdi:weather-windy");  
    sendDiscovery("wind_gust", "m/s", "wind_speed", "mdi:weather-windy-variant");  
    sendDiscovery("wind_direction", "°", nullptr, "mdi:compass");  
    sendDiscovery("rain", "mm", "precipitation", "mdi:weather-rainy");  
    sendDiscovery("outdoor_battery", nullptr, "battery", "mdi:battery");  
    sendDiscovery("forecast", nullptr, nullptr, "mdi:weather-partly-cloudy");  
    sendDiscovery("rssi", "dBm", "signal_strength", "mdi:signal");  
    sendDiscovery("freq", "MHz", nullptr, "mdi:radio-tower");
```

```

    sendDiscovery("bmp_temperature", "°C", "temperature", "mdi:thermometer");
    sendDiscovery("bmp_pressure", "hPa", "pressure", "mdi:gauge");
    sendDiscovery("bmp_pressure_mmhg", "mmHg", "pressure", "mdi:gauge");
    Serial.println("[MQTT] Universal Home Assistant sensors registered successfully.");
}

void publishMQTTData() {
    if (!mqtt.connected()) return;

    StaticJsonDocument<1024> doc;
    char out_buf[1024];

    doc["temperature"] = temperature;
    doc["humidity"] = humidity;
    doc["pressure"] = pressure;
    doc["pressure_mmhg"] = pressure * 0.750062;
    doc["wind_speed"] = wind_avg_smooth;
    doc["wind_gust"] = wind_gust_max;
    doc["wind_direction"] = wind_dir_avg;
    doc["rain"] = rain;
    doc["outdoor_battery"] = outdoor_battery;
    doc["sensor_id"] = sensor_id;
    doc["forecast"] = zambretti_forecast;
    doc["rssi"] = last_rssi;
    doc["freq"] = current_freq;
    doc["bmp_temperature"] = bmp_temperature;
    doc["bmp_pressure"] = bmp_pressure_raw + bmp_calibration_offset;
    doc["bmp_pressure_mmhg"] = (bmp_pressure_raw + bmp_calibration_offset) * 0.750062;
    doc["radio_errors"] = radio_errors;
    doc["radio_success"] = radio_success;
    doc["uptime"] = millis() / 1000;
    if (rtc_found) doc["datetime"] = formatDateTime(rtc.now());

    memset(out_buf, 0, sizeof(out_buf));
    serializeJson(doc, out_buf, sizeof(out_buf));
    mqtt.publish(("smart/" + hostname + "/state").c_str(), out_buf, false);
}

void broadcastMQTTData() {
    if (mqtt.connected()) {
        publishMQTTData();
        lastMQTTPublish = millis();
    }
}

```

```

void reconnectMQTT() {
    if (mqtt_ip.length() < 7) return;
    String clientId = "WeatherGate_" + hostname + "_" + String((unsigned long)ESP.getEfuseMac(), HEX);
    if (mqtt.connect(clientId.c_str(), mqtt_user.c_str(), mqtt_password.c_str())) {
        Serial.println("[MQTT] Connected to broker.");
        mqtt_connected = true;
        if (!discovery_sent) {
            registerAllSensors();
            discovery_sent = true;
        }
    } else {
        Serial.printf("[MQTT] Connection fault, rc=%d\n", mqtt.state());
        mqtt_connected = false;
    }
}

// ===== НИЗКОУРОВНЕВЫЙ ОБРАБОТЧИК ДАТЧИКА BMP280 =====
void readBMP280() {
    float p_raw = bmp.readPressure() / 100.0F;
    if (p_raw < 850.0 || p_raw > 1100.0) return;
    bmp_pressure_raw = p_raw;
    bmp_temperature = bmp.readTemperature();

    float calibrated = bmp_pressure_raw + bmp_calibration_offset;
    pressure = calibrated + (altitude_meters * 0.12);

    static unsigned long lastBMPLog = 0;
    if (millis() - lastBMPLog > 60000) {
        Serial.printf("[BMP280] Sensor values: P=%.1f hPa, T=%.1f°C\n", pressure, bmp_temperature);
        lastBMPLog = millis();
    }
}

// ===== RTL433 CALLBACK (ЯДРО CORE 0) =====
void rtl433_Callback(char* message) {
    StaticJsonDocument<512> doc;
    DeserializationError error = deserializeJson(doc, message);
    if (error) return;
    const char* model = doc["model"];
    if (model && strstr(model, "FineOffset")) {
        WeatherMessage msg;
        msg.station_id = doc["id"] | 0;
        msg.battery_ok = ((doc["battery_ok"] | doc["battery_OK"] | 0) == 1);
    }
}

```

```

msg.outdoor_temp = doc["temperature_C"] | 0.0;
msg.outdoor_humidity = doc["humidity"] | 0;
msg.wind_dir_deg = doc["wind_dir_deg"] | 0;
msg.wind_speed_ms = doc["wind_avg_m_s"] | 0.0;
msg.wind_gust_ms = doc["wind_max_m_s"] | 0.0;
msg.rain_mm = doc["rain_mm"] | 0.0;

// ИСПРАВЛЕНИЕ: Безопасное чтение RSSI из прилетевшего JSON манифеста
msg.rssi = doc["rssi"] | -105.0;
msg.current_freq = current_working_frequency;
msg.data_valid = true;

if (weatherQueue) {
    xQueueSend(weatherQueue, &msg, 0);
}
}

// ===== СЛУЖЕБНЫЙ СЛОЙ И СЕТЕВЫЕ СОБЫТИЯ ETHERNET =====
void WiFiEvent(WiFiEvent_t event) {
    switch (event) {
        case ARDUINO_EVENT_ETH_START:
            Serial.println("[ETH] Ethernet Driver Started.");
            ETH.setHostname(hostname.c_str());
            break;
        case ARDUINO_EVENT_ETH_CONNECTED:
            Serial.println("[ETH] Hardware Link Established.");
            break;
        case ARDUINO_EVENT_ETH_GOT_IP:
            Serial.printf("[ETH] Core DHCP Server assigned IP: %s\n", ETH.localIP().toString().c_str());
            eth_connected = true;
            break;
        case ARDUINO_EVENT_ETH_DISCONNECTED:
            Serial.println("[ETH] Link Broken!");
            eth_connected = false;
            break;
        default: break;
    }
}

// ===== СТАБИЛЬНЫЙ TASK РАДИОМОДУЛЯ ДЛЯ CORE 0 =====
// ===== ИСПРАВЛЕННЫЙ TASK РАДИОМОДУЛЯ ПО СТАТИЧЕСКИМ СИГНАТУРАМ =====
void radioTask(void *pvParameters) {

```

```

Serial.printf("[CORE 0] High-priority Radio task deployed on core %d\n", xPortGetCoreID());

// ИСПРАВЛЕНИЕ: Выделяем память и физически рождаем объект rtl_433_ESP в куче!
// Это происходит спустя 6 секунд после старта платы, когда Ethernet уже на 100% запущен
rf = new rtl_433_ESP();

// Инициализируем приёмник через штатные статические методы класса библиотеки
rtl_433_ESP::initReceiver(PIN_GDO0, 915.00);

// Передаем символьный буфер, привязывая наш отладочный коллбек
static char localMsgBuffer[512]; // Явно задан размер массива
rf->setCallback(rtl433_Callback, localMsgBuffer, sizeof(localMsgBuffer));

// Включение приемника
rtl_433_ESP::enableReceiver();

current_working_frequency = 915.00;
Serial.printf("[CORE 0] Transceiver operational at %.2f MHz\n", current_working_frequency);

for (;;) {
    rf->loop(); // Вызов через стрелочную нотацию указателя кучи
    vTaskDelay(pdMS_TO_TICKS(1));
}
}

// ===== ЖЕСТКО ВЫРОВНЕННАЯ ИНИЦИАЛИЗАЦИЯ (SETUP) =====
void setup() {
    Serial.begin(115200);
    delay(200); // Даем железу физически проснуться при подаче питания
    printResetReason();

    // ЭТАП 1: ТОТАЛЬНЫЙ ПРИОРИТЕТ СЕТИ. Запускаем Ethernet в кристальной пустоте!
    // Никаких I2C, никаких радио-вызовов. Даем LAN8720 захватить GPIO 5 для CLK 50MHz.
    WiFi.mode(WIFI_OFF);
    btStop();
    WiFi.onEvent(WiFiEvent);

    Serial.println("[SYS] Boot stage 1: Deploying industrial hardware Ethernet...");
    ETH.begin(ETH_PHY_LAN8720, 1, 16, 23, 18, ETH_CLOCK_GPIO0_IN);

    // ЭТАП 2: Даем роутеру 6 секунд в полной тишине, чтобы он выдал IP по DHCP
    Serial.println("[SYS] Boot stage 2: Waiting 6s for DHCP lease before turning on I2C/Radio...");
    delay(6000);
}

```

```

// ЭТАП 3: Инициализация файловой системы LittleFS и NVS
if (LittleFS.begin(true)) {
    Serial.println("[FS] Static LittleFS container ready.");
    loadSettings(); // Загружаем параметры РАДИОМОДУЛЯ строго СЕЙЧАС, когда сеть уже работает!
} else {
    Serial.println("[ERR] LittleFS mount crash!");
}

// ЭТАП 4: Безопасный запуск I2C шин, когда GPIO 5 уже отдан под нужды MAC-адреса Ethernet
Serial.println("[SYS] Boot stage 3: Securing I2C channels...");
I2C_Clock.begin(PIN_I2C0_SDA, PIN_I2C0_SCL, 100000);
I2C_Bmp.begin(PIN_I2C1_SDA, PIN_I2C1_SCL, 100000);

// Инициализация барометра BMP280
if (bmp.begin(0x76)) {
    Serial.println("[BMP280] Industrial I2C barometer identified.");
} else {
    Serial.println("[ERR] BMP280 lost from hardware I2C channel 1!");
}

// ЭТАП 5: Запуск сетевых сокетов Веб-Сервера
if (mqtt_ip.length() > 5) {
    mqtt.setServer(mqtt_ip.c_str(), 1883);
    mqtt.setCallback(mqttCallback);
    mqtt.setKeepAlive(60);
}

initWebServer();
server.on("/", HTTP_GET, []() { if (!handleFileRead("/")) server.send(404, "text/plain", "Static error"); });
server.begin();

// ЭТАП 6: Сброс системных таймеров истории uPlot
lastSaveTime = millis();
lastHistoryTime = millis();
lastBMPRead = millis();
readBMP280(); // Первичный безопасный опрос датчика давления

weatherQueue = xQueueCreate(10, sizeof(WeatherMessage));

// ЭТАП 7: Динамический запуск радио-декодера FreeRTOS на ядре Core 0
xTaskCreatePinnedToCore(radioTask, "RadioTask", 10240, NULL, 3, NULL, 0);

Serial.println("[SYS] DevOps Hardware Boot Sequence Complete. All channels operational.");
}

```

```
// ===== ГЛАВНЫЙ ЦИКЛ ОБРАБОТКИ ДАННЫХ (CORE 1) =====

void loop() {
    server.handleClient();

    if (millis() - lastBMPRead >= BMP_READ_INTERVAL) {
        lastBMPRead = millis();
        readBMP280();
    }

    WeatherMessage msg;
    if (weatherQueue && xQueueReceive(weatherQueue, &msg, 0) == pdPASS) {
        temperature = msg.outdoor_temp;
        humidity = msg.outdoor_humidity;
        wind_avg_smooth = msg.wind_speed_ms;
        wind_gust_max = msg.wind_gust_ms;
        wind_dir_avg = msg.wind_dir_deg;
        rain = msg.rain_mm;
        sensor_id = msg.station_id;
        last_rssi = msg.rssi;
        current_freq = msg.current_freq;
        outdoor_battery = msg.battery_ok ? "OK" : "LOW";
        radio_success++;

        if (temperature < temp_min || temp_min == 0) temp_min = temperature;
        if (temperature > temp_max) temp_max = temperature;
        if (wind_avg_smooth > wind_max) wind_max = wind_avg_smooth;
        if (rain > rain_max) rain_max = rain;

        updateWindAverage(wind_dir_avg);
        updateWindSpeedAverage(wind_avg_smooth, wind_gust_max);

        Serial.printf("[WS1080] Frame pulled: T=%.1f C, H=%u%%, RSSI=%.1f dBm\n", temperature, humidity, last_rssi);
        broadcastMQTTData();
    }

    if (eth_connected) {
        if (mqtt.connected()) {
            mqtt.loop();
        } else if (millis() - lastMQTTReconnect >= MQTT_RECONNECT_INTERVAL) {
            lastMQTTReconnect = millis();
            reconnectMQTT();
        }
    }
}
```



```

static uint32_t lastPeriodic = millis();
if (millis() - lastPeriodic >= 10000) {
    lastPeriodic = millis();
    periodicSave();
}

if (millis() - lastHistoryTime >= HISTORY_INTERVAL && radio_success > 0) {
    lastHistoryTime = millis();
    addHistory(temperature, pressure, wind_avg_smooth, rain);
    updatePressureHistory(pressure);
    calculateZambrettiForecast();
}

if (scan_active) {
    updateAutoScanner();
}
delay(1);
}

// ===== РАЗДАЧА СЖАТЫХ СТАТИЧЕСКИХ ФАЙЛОВ ИНТЕРФЕЙСА (GZIP) =====
bool handleFileRead(String path) {
    if (path.endsWith("/")) path += "index.html";

    String contentType = "text/plain";
    if (path.endsWith(".html")) contentType = "text/html";
    else if (path.endsWith(".js")) contentType = "text/javascript";
    else if (path.endsWith(".css")) contentType = "text/css";

    String gzPath = path + ".gz";
    if (LittleFS.exists(gzPath)) {
        File file = LittleFS.open(gzPath, "r");
        if (file) {
            server.streamFile(file, contentType);
            file.close();
            return true;
        }
    }
    if (LittleFS.exists(path)) {
        File file = LittleFS.open(path, "r");
        if (file) {
            server.streamFile(file, contentType);
            file.close();
            return true;
        }
    }
}

```

```

    }
}
return false;
}

// ===== ОБРАБОТЧИК ВХОДЯЩИХ КОМАНД MQTT =====
void mqttCallback(char* topic, byte* payload, unsigned int length) {
    // Реактивное погодное ядро работает на чтение радиоэфира и трансляцию в НА,
    // но коллбек должен присутствовать для удержания сетевой структуры PubSubClient.
    String message = "";
    for (unsigned int i = 0; i < length; i++) message += (char)payload[i];
    Serial.printf("[MQTT RECV] Topic: %s, Message: %s\n", topic, message.c_str());
}

// ===== ВЫВОД ПРИЧИНЫ ПЕРЕЗАГРУЗКИ ПРОЦЕССОРА =====
void printResetReason() {
    esp_reset_reason_t reason = esp_reset_reason();
    Serial.print("[SYS] Reset reason: ");
    switch (reason) {
        case ESP_RST_POWERON: Serial.println("POWERON_RESET"); break;
        case ESP_RST_EXT:     Serial.println("External pin reset"); break;
        case ESP_RST_SW:      Serial.println("Software reset via esp_restart"); break;
        case ESP_RST_PANIC:   Serial.println("Software exception/panic"); break;
        case ESP_RST_INT_WDT:  Serial.println("Interrupt watchdog reset"); break;
        case ESP_RST_TASK_WDT: Serial.println("Task watchdog reset"); break;
        case ESP_RST_WDT:     Serial.println("Other watchdogs reset"); break;
        case ESP_RST_DEEPSLEEP: Serial.println("Deep sleep reset"); break;
        case ESP_RST_BROWNOUT: Serial.println("Brownout reset (voltage drop)"); break;
        case ESP_RST_SDIO:    Serial.println("SDIO reset"); break;
        default:              Serial.println("Unknown reset cause"); break;
    }
}
}

```